# CFGs and ASTs

CS/COE 1622

Jarrett Billingsley

# Class Announcements

- did you start the project yet?
  - a fair number of you haven't accepted the assignment on Github...
  - do iiiiiiit >:V

# Beyond Regular Grammars

# Regular?

- lexing grammars describe **regular languages.** what's that mean?

```
Program: (WS? Token)* WS? '<eof>'
WS:      ' ' | '\t' | '\n'
Token:   '(' | ')' | Id | IntLit
Id:      IdStart IdCont*
IdStart: <alphabetic> | '_'
IdCont:  IdStart | <digit>
IntLit:  <digit>+
```

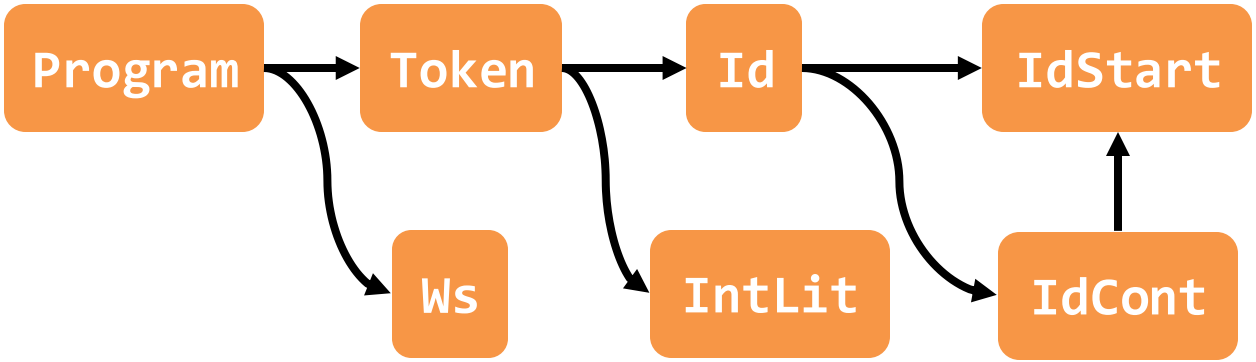
it's a little nonobvious, but if we start at the start rule, **Program**, the right-hand sides of every rule **never take us in a loop to the *same rule twice.***

this puts strict limitations on the **kinds of strings** these grammars can recognize…

…but it *does* make these languages easier to reason about and implement.

# Another way to look at it

- if we look at the **dependency graph** of these rules:




now it becomes more obvious that
there are **no cycles in this graph.**

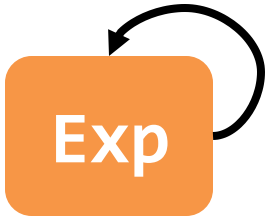
(this is called a **DAG:** a directed acyclic graph.
it's like a tree, but nodes can share parents.)

but… what happens if we remove that requirement?

# A whole new world

- let's say our **alphabet** is `A = { '(', ')', 'e' }`.
- and our grammar rule is `Exp: 'e' | '(' Exp ')'`
- valid strings would be `e`, `(e)`, `((e))`, etc. for **any** number of parens.

one **l o o p y b o i** is all it takes to give us this power.




but power comes at a cost: this kind of language is **strictly more complex** than regular languages and requires **more complex algorithms** to implement.

once you allow these kinds of recursive rules, you are now in the land of **context-free languages!**

# So what can they do?

- context-free grammars (**CFGs**) can represent **nesting** of any depth.
- this happens to map very nicely onto our intuition (instinct?) about how language works in general.

in **human languages,** we can nest phrases inside other ones.

the cat (on the bed (in that room))

with CFGs, we can nest expressions…

```
f(x * (y + 3))
```

and statements!

```
if x == 10 {
  if y == 20 {
    f();
  }
}
```

# But what *can't* they do?

- there are *a lot* of kinds of languages they can't represent.
- but for most ***computer*** languages, this isn't really a problem.
- unlike natural human languages, computer languages…
  - are **designed up front,** rather than emerging naturally
  - are meant to be **unambiguous**
  - are meant to be **easy to read and maintain**
- so something simpler than human languages is a good thing.
- THERE'S STILL THIS THING THOUGH

```
int y = x >> 3;
List<List<String>> l;
```

to properly parse this requires a **context-sensitive grammar (CSG)…** but the ambiguity here is simple enough that we can fake it using a CFG and some parsing kludges.

# ASTs

# From lexical to syntactic

- the lexer was responsible for splitting the source into "words."
- the **parser** looks at those words and extracts the **syntactic structure** from them, building a tree representation of your program.



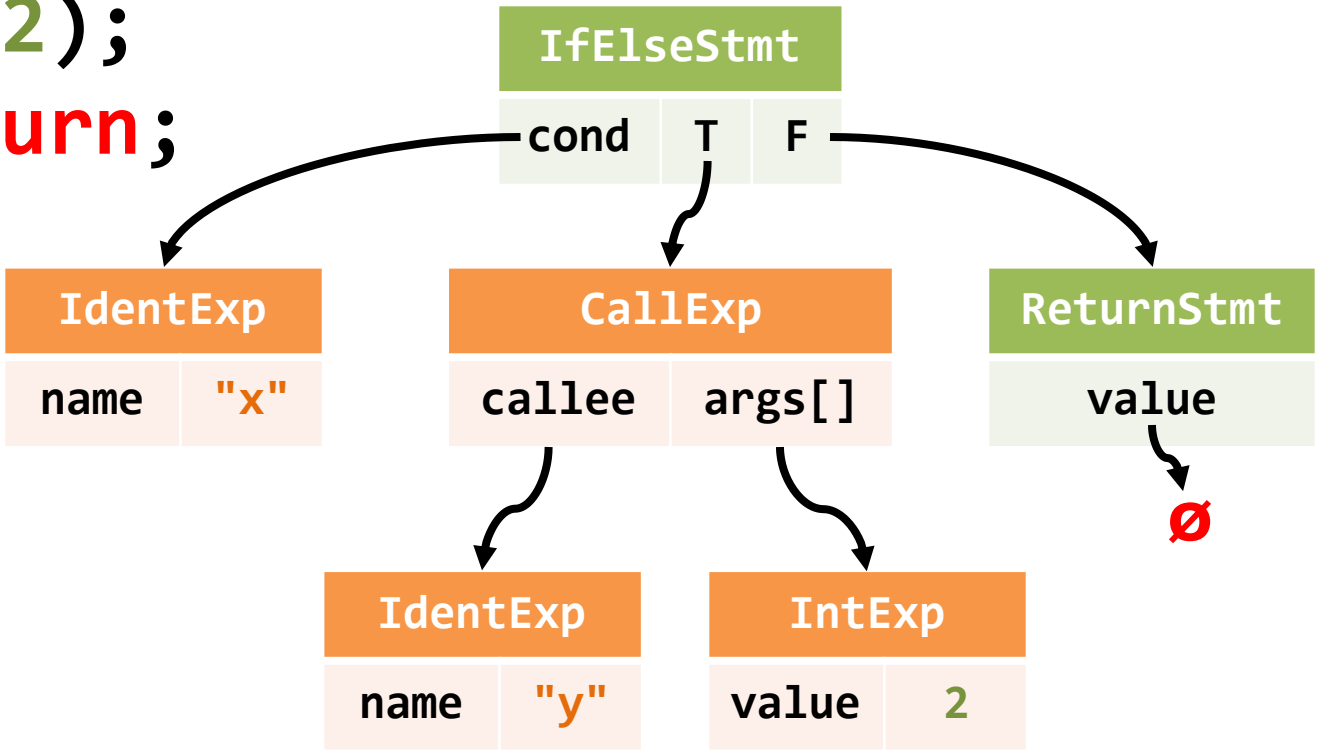

the parser typically implements a CFG whose **alphabet** (the set of terminals) is the **tokens** that were produced by the parser.

in doing so it checks your **syntax –** to make sure you wrote something that, well, *looks like* a program.

# A tree of the syntax, which is abstract

- the **abstract syntax tree (AST)** is the structure the parser builds.
- trees are much easier to work with than lists of characters or tokens.

```
if(x) y(2);
else return;
```



# Why is it called "abstract"?

- because we can **change the syntax of our language** but the AST representation doesn't have to change. it **decouples** them.

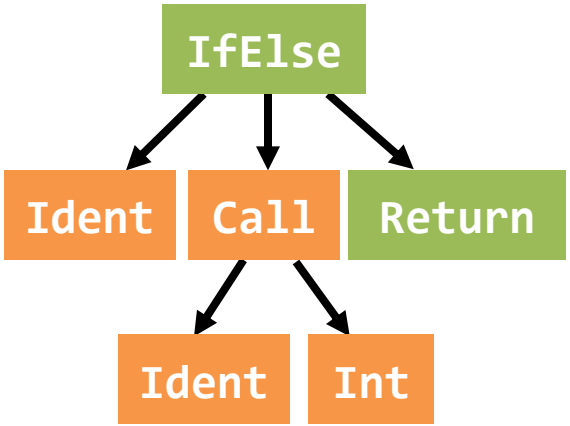
**Java/C**
```
if(x) y(2); else return;
```

**Rust**
```
if x { y(2); } else { return }
```

**Lua**
```
if x then y(2) else return end
```

**Ruby**
```
if x
  y 2
else
  return
endif
```

**Python**
```
if x:
    y(2)
else:
    return
```




of course, these languages *work* differently, but the point is that the *syntax* is just fluff.

# Values, expressions, and statements

- computation is the act of transforming **values** into different values.
  - values are just things like ints, strings, objects, etc.

when you **execute an expression,** you get a value.

```
x + y
```

```
2.8
```

```
sin(2 * ang)
```

```
arr[i] + ";"
```

```
obj.field
```

when you **execute a statement,** you... don't.

```
return;
```

```
if(…) …
else …
```

```
printf("hi!\n");
```

```
{ …statements… }
```

but there are many exceptions, and not every language makes this distinction.

it's good to know the terms though, cause they come up a lot in parsing and semantics.

# What about Rust?

- actually, Rust **only has expressions.** *everything* can give a value.

```
let x = if y { 10 } else { 20 };

fn radix(c: char) -> Base {
  println!("in radix");
  match c {
    'x' | 'X' => Base::Hex,
    'b' | 'B' => Base::Bin,
    _         => Base::Dec,
  }
}
```

this demonstrates a sort of confusing rule: you can omit `return` if you're in the **last expression in the function.**

this `match` is the last expression, so whatever value it gives becomes the return value!

but this is confusing, so I won't use it this term. :)

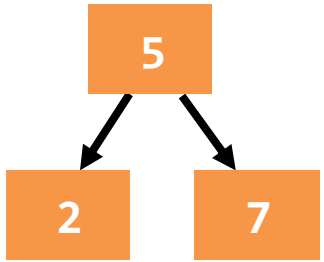
# Trees in Rust

# In Java...

- you may have learned about trees (or their one-dimensional cousins, linked lists) and seen them written like this:

```
class Node<T> {
  Node<T> left, right;
  T value;
  Node(T t) { value = t; }
}
```

```
Node<Integer> a = new Node<>(5);
a.left  = new Node<>(2);
a.right = new Node<>(7);
```




**`left`** and **`right`** have two special properties that make this representation work:

1. they are **references:** they refer *indirectly* to **`Node`** objects.
2. they can optionally hold **`null`** to indicate that they are not pointing to anything.

# Doing it in Rust?

- if we try to do this, we get an error:

```
struct Node<T> {
  left:  Node<T>,
  right: Node<T>,
  value: T,
}
```

it says this type has **infinite size,** but it gives us a hint:

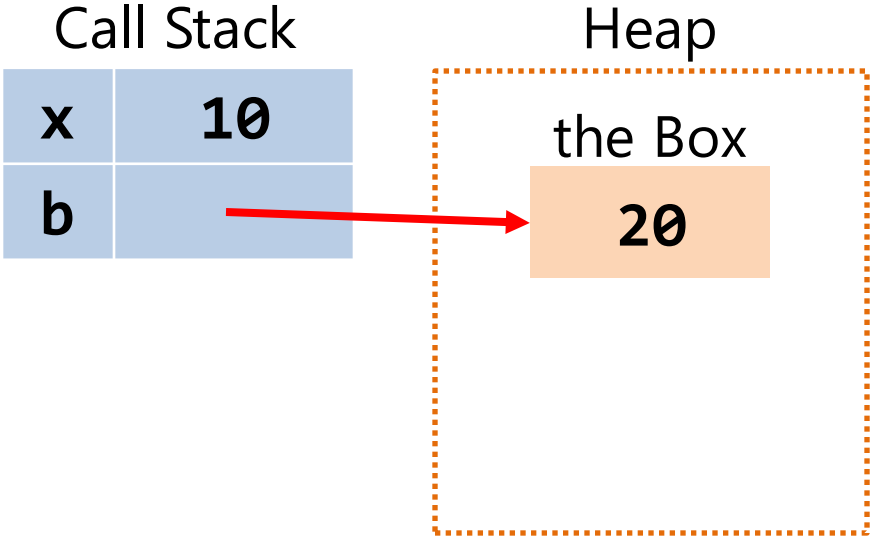
```
struct Node<T> {
  left:  Box<Node<T>>,
  right: Box<Node<T>>,
  value: T,
}
```

great, it compiles! *what the heck does it mean tho*

# I would not eat them with a fox

- a **Box** is the simplest kind of reference in Rust: it allocates an object **on the heap** (just like Java's **new**) and points to it indirectly.
- you create a boxed object with **`Box::new()`**:

```
let x = 10;              // an int on the stack
let b = Box::new(20); // an int on the heap
```




I mean, normally you wouldn't box an int, but it's just for the example.

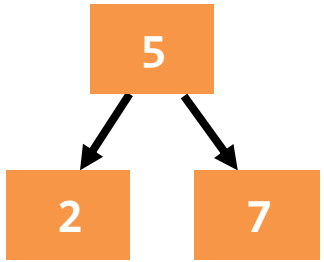
okay, let's make a **`Node<T>`** then!

# Curses! Foiled again!

- when you make a struct, you have to give values for **all** the fields.

```
struct Node<T> {
  left:  Box<Node<T>>,
  right: Box<Node<T>>,
  value: T,
}
```

```
let a = Node {
  value: 5,
  left: Box::new(Node {
    value: 2,
    left:  ……uhhhhhhhh
```




what do we write for **left,** here? 2 *has* no left child.

what we've made here is a type that is *impossible to construct!*

so we need something to satisfy the other important property of the Java **`Node<T>`** variables: the possibility of being **`null`**.

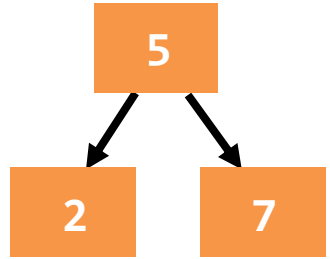
the solution is to use **`Option<Box<Node<T>>>`**.

# Option<Box<Node<T>>>? That's... verbose

- a little, yeah. but this comes up less often than you'd think.
- add a constructor and it becomes easy to use:

```
struct Node<T> {
  left:  Option<Box<Node<T>>>,
  right: Option<Box<Node<T>>>,
  value: T,
}
```

```
let mut a = Node::new(5);
a.left  = Some(Node::new(2));
a.right = Some(Node::new(7));
```




and now we have something very similar to the original Java, except we have to use **Some(..)** to indicate the opposite of **None** (aka **null**).

this code is in the **rust_trees** example.

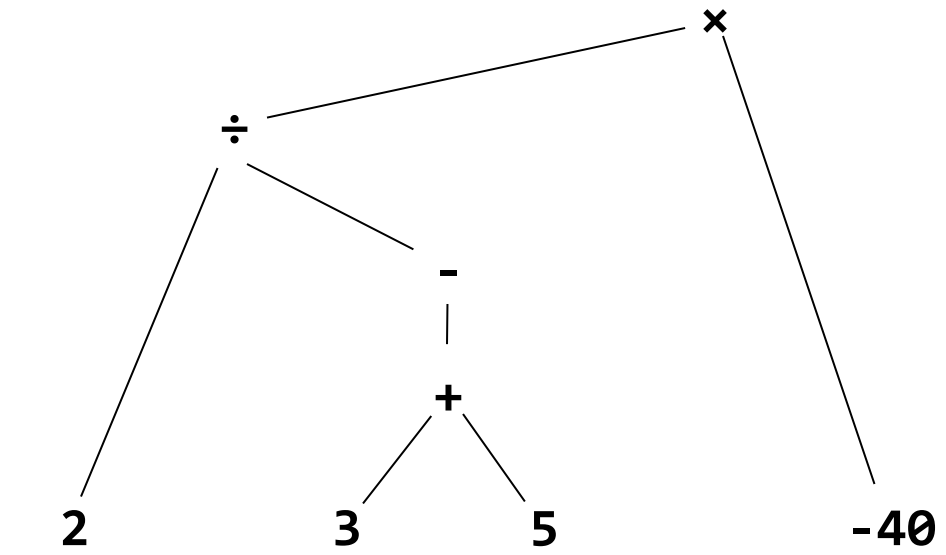
# A brief digression on Option

- let's look at another example, **`rust_option`**!
- **`Option`** gives you something like Java's **`null`**, but it's *opt-in*, and works on *any* type, not just object references.
- there are a few ways of "getting the value out of" an **`Option`**, but…
  - **`.unwrap()`** is the most risky, cause it could crash your program!

# Defining and using an AST

# Before we get to a real language...

- let's start with something simple.
- maybe we're writing some calculator software and want to represent what the user types in *symbolically,* instead of just evaluating it.




how we get from what was typed in at the bottom to the tree up top is a topic for an upcoming lecture, but...

if the expression is a tree, we can have fun with it!

# Representing this tree

- we've got a few kinds of AST node here:
  - **constants** (`2`, `3`, `5`, `-40`)
  - **negation** (`-x`)
  - **addition, subtraction, multiplication, division**
    - we can group these under a single kind: **binary operators**
    - not binary as in the base, binary as in "has two operands"
- when you have **choices** of a type, the thing to use is an **enum.**
- this enum appears in the **`ast_math`** example:

```
enum AstNode {
  Const  { val: f64 },
  Negate { lhs: Box<AstNode> },
  Binary { op: BinOp, lhs: Box<AstNode>, rhs: Box<AstNode> },
}
```

look, no **`Option`**s!

# Recursion Refresher

- **recursion** is a programming technique where a function calls itself.
- there is an important fact about recursion that a lot of people are hazy on, even as juniors and seniors:

local variables are ***duplicated*** **for each recursive call.**

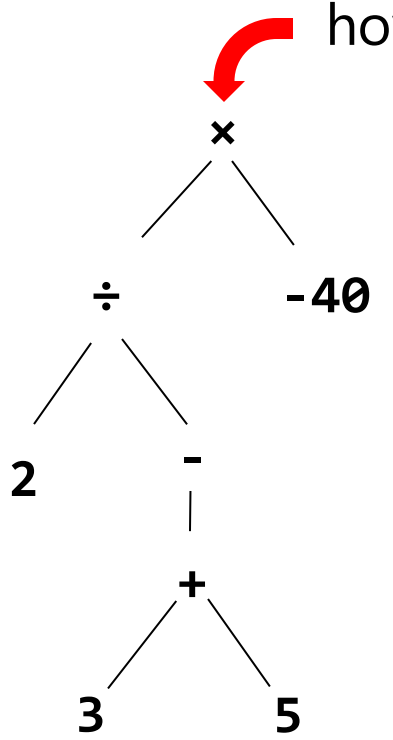
```
fn sum(x: i32) -> i32 {
 if x <= 1 {
  return x;
 } else {
  return x + sum(x - 1);
 }
}
```

if I call **sum(5)**, every recursion gets its *own copy of x,* and the in-progress recursions do not "share" that variable.

| | Stack |
|---|---|
| *sum(5)* | x = 5 |
| *sum(4)* | x = 4 |
| *sum(3)* | x = 3 |
| *sum(2)* | x = 2 |
| *sum(1)* | x = 1 |

# Trees ♥ Recursion

- forget Fibonacci: **recursion is *exactly* what you need for trees.**
- we have this tree, and a variable pointing to the root.




how do you evaluate a multiplication?

1. evaluate the left-hand side (**LHS**)
2. evaluate the right-hand side (**RHS**)
3. multiply them together

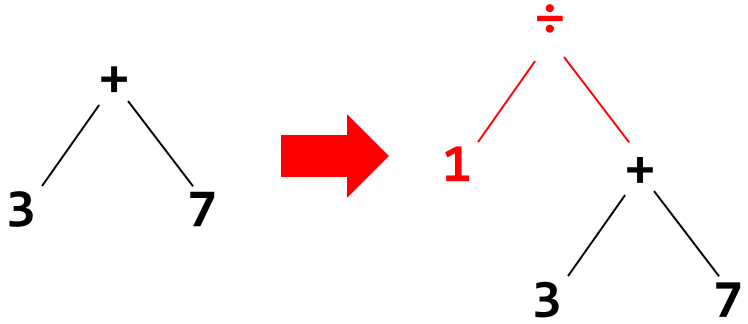
the first two steps are done as **recursive calls** to evaluate the two children.

that's it. that's how you write a recursive algorithm over a tree. (see **`AstNode::eval`**)

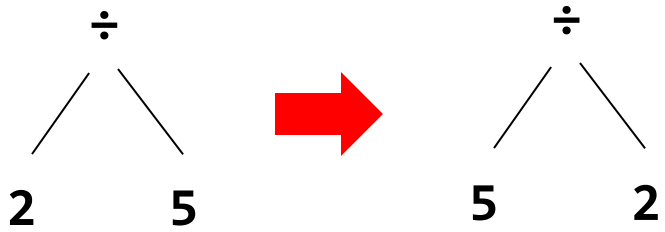
# What else could you do with them?

- you could **apply transformations** to the tree.
- maybe we have a calculator feature to get the **reciprocal.**

doing "one over" means adding two new nodes and making the division the root.




of course, if the input *is* a division, we can special-case it.




this is what **`AstNode::recip`** in the example does!

# What if the *programmer* could do that?

- some languages have **macros:** things that *look* like functions, but which operate **on the AST at compile-time.**
  - that's really what **`println!()`** is!
  - the details of how **`println!()`** works are beyond the scope of this lecture but that's why it yells: it's a *macro,* not a function. it generates different ASTs based on what arguments you give it.
- if you've experienced C's *preprocessor macros* (**`#define`**), those are similar, but they operate on **tokens** instead of the AST.
  - still, you can do some impressive stuff with them!

# Ok, but, how does the AST get built

- uhhhhhhhhhhhhhhh wellll
  - parsing!!!!!!
  - parsing is definitely more complicated than lexing
  - which is why we have the next *two* lectures dedicated to it.
- but that's all for today. good luck on the project, have a nice break, and **see you in a week!**